PYTHON PRACTICALS

Simple Programs with Sample Outputs

Based on the 8 practicals shown in your syllabus image. Each practical includes easy-to-understand code and the output you should see when you run it.

Platform: Jupyter Notebook / Google Colab

Tip: In Jupyter Notebook or Google Colab, paste each program into a separate code cell and press Shift + Enter to run it. Outputs below are sample outputs.

1. Install/Use Jupyter Notebook or Google Colab — Basic Operations

Program:

```
# Run this in a code cell
print("Hello, Python!")

name = "Pratham"
print("Welcome", name)

# Markdown cell example (type this in a Markdown cell, not a code cell):
# ## My First Notebook
# This notebook runs Python code.
```

Sample Output:

```
Hello, Python!
Welcome Pratham
```

Note: Google Colab works in a browser. For local Jupyter, install it with: pip install notebook, then run: jupyter notebook.

2. Load a CSV File Using Pandas and Display First/Last Records

Program:

```
import pandas as pd
from io import StringIO

csv_data = """Name,Age,City
Asha,20,Pune
Rahul,21,Mumbai
Neha,19,Jalgaon
Amit,22,Nashik
"""

df = pd.read_csv(StringIO(csv_data))

print("First 2 records:")
print(df.head(2).to_string(index=False))

print("\nLast 2 records:")
print(df.tail(2).to_string(index=False))
```

Sample Output:

```
First 2 records:
 Name  Age   City
 Asha   20   Pune
Rahul   21 Mumbai

Last 2 records:
 Name  Age    City
 Neha   19 Jalgaon
 Amit   22  Nashik
```

Note: This example uses a small CSV string so it runs without downloading a dataset. For a CSV file on your computer, use pd.read_csv('filename.csv').

3. Explore a Dataset Using info(), describe(), and Column Data Types

Program:

```
import pandas as pd
from io import StringIO

csv_data = """Name,Age,Marks
Asha,20,85
Rahul,21,78
Neha,19,92
"""

df = pd.read_csv(StringIO(csv_data))

print("Dataset information:")
df.info()

print("\nColumn data types:")
print(df.dtypes)

print("\nStatistical summary:")
print(df.describe().round(2))
```

Sample Output:

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   Marks   3 non-null      int64
dtypes: int64(2), object(1)
memory usage: about 200 bytes

Column data types:
Name     object
Age       int64
Marks     int64

Statistical summary:
         Age  Marks
count   3.00   3.00
mean   20.00  85.00
std     1.00   7.00
min    19.00  78.00
25%    19.50  81.50
50%    20.00  85.00
75%    20.50  88.50
max    21.00  92.00
```

Note: Exact memory-usage text shown by info() can vary slightly by Pandas version. The key points are the row count, column names, non-null values, and data types.

4. Operations on Strings, Lists, Tuples, Dictionaries, and Sets

Program:

```
# String
text = "Python"
print("String:", text)
print("First character:", text[0])
print("Slice:", text[1:4])

# List (changeable)
numbers = [10, 20, 30, 40]
print("List item:", numbers[1])
print("List slice:", numbers[1:3])

# Tuple (not changeable)
colors = ("red", "blue", "green")
print("Tuple item:", colors[0])

# Dictionary (key-value pairs)
student = {"name": "Asha", "age": 20}
print("Student name:", student["name"])

# Set (unique values)
unique_numbers = {1, 2, 2, 3}
print("Set:", sorted(unique_numbers))
```

Sample Output:

```
String: Python
First character: P
Slice: yth
List item: 20
List slice: [20, 30]
Tuple item: red
Student name: Asha
Set: [1, 2, 3]
```

Note: Indexes start from 0. A slice such as [1:4] includes index 1 but stops before index 4.

5. Numerical Operations Using math and random Modules

Program:

```
import math
import random

number = 25
print("Square root:", math.sqrt(number))
print("Factorial:", math.factorial(5))
print("Power:", math.pow(2, 3))

# Set a seed so this example gives a repeatable result
random.seed(1)
print("Random integer:", random.randint(1, 10))
```

Sample Output:

```
Square root: 5.0
Factorial: 120
Power: 8.0
Random integer: 3
```

Note: random.seed(1) makes the sample random result repeatable in standard Python. Without a seed, the random integer may change.

6. File Handling — Read, Write, and Append

Program:

```
# Write text to a file
with open("sample.txt", "w") as file:
    file.write("Hello, file!\n")

# Append another line
with open("sample.txt", "a") as file:
    file.write("This is the second line.\n")

# Read the file
with open("sample.txt", "r") as file:
    content = file.read()

print(content, end="")
```

Sample Output:

```
Hello, file!
This is the second line.
```

Note: The file is created in the current working folder. Mode 'w' writes/overwrites, 'a' appends, and 'r' reads.

7. Exception Handling Using try-except

Program:

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print("Result:", result)
except ValueError:
    print("Please enter a valid whole number.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Sample Output:

```
Example 1:
Enter a number: 2
Result: 5.0

Example 2:
Enter a number: 0
Cannot divide by zero.

Example 3:
Enter a number: abc
Please enter a valid whole number.
```

Note: Only one example is run each time. The except blocks prevent the program from stopping with an error for these cases.

8. Create NumPy Arrays and Perform Indexing and Slicing

Program:

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
print("Array:", arr)
print("First item:", arr[0])
print("Last item:", arr[-1])
print("Slice:", arr[1:4])

matrix = np.array([[1, 2, 3], [4, 5, 6]])
print("2D array:")
print(matrix)
print("Item at row 2, column 3:", matrix[1, 2])
```

Sample Output:

```
Array: [10 20 30 40 50]
First item: 10
Last item: 50
Slice: [20 30 40]
2D array:
[[1 2 3]
 [4 5 6]]
Item at row 2, column 3: 6
```

Note: Install NumPy if needed with: pip install numpy. In Google Colab, NumPy is usually already available.

9. Create Line, Bar and Histogram Plots Using Matplotlib

Program: This simple program creates three plots using sample data. No CSV file is needed.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 15, 25]

plt.plot(x, y)
plt.title("Line Plot")
plt.show()

plt.bar(x, y)
plt.title("Bar Plot")
plt.show()

plt.hist(y)
plt.title("Histogram")
plt.show()
```

Output: Three graphs appear one by one: a line plot, a bar plot, and a histogram.

Note: Install Matplotlib if needed: `pip install matplotlib`

10. Create Line, Bar and Histogram Plots Using Seaborn

Program: Seaborn works with Matplotlib and uses the same sample data. No CSV file is needed.

```
import seaborn as sns
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 15, 25]

sns.lineplot(x=x, y=y)
plt.title("Line Plot")
plt.show()

sns.barplot(x=x, y=y)
plt.title("Bar Plot")
plt.show()

sns.histplot(y)
plt.title("Histogram")
plt.show()
```

Output: Three graphs appear one by one: a line plot, a bar plot, and a histogram.

Note: Install Seaborn if needed: `pip install seaborn`

Run in Jupyter: Paste each practical into a code cell and press Shift + Enter. If the libraries are missing, install them in a separate cell using `!pip install matplotlib seaborn`.